

数据结构与算法分析

第二章 线性表

线性表是一种最常用最简单的数据结构（数据类型相同），一个线性表是n个元素的有限序列。

顺序表

顺序表是一种随机存储结构，顺序存储结构下读第i个元素，取i个元素的前驱或后继容易实现。

数据类型

```
# define MAXSIZE 1024
typedef char datatype;
typedef struct {
    datatype data[MAXSIZE];
    int last;
} sequenlist
```

创建顺序表

```
sequenlist *Create(){
    int i = 0;char ch;
    L=(sequenlist *)malloc(sizeof(sequenlist));
    L->last = -1;
    printf("请输入顺序表L中的元素，以'#'结束\n");
    while(ch=getche()!='#'){
        L->data[i++] = ch;
        L->last++;
    }
    return L;
}
```

按位插入

```
int Insert(sequenlist *L,int x,int i){
    int j;
    if((L->last)>=MAXSIZE-1){
        printf("overflow\n");
        return 0;
    }else{
        if((i<1)|| (i>(L->last)+2)){
            printf("error\n");
            return 0;
        }else{
            for (j=L->last;j>=i-1;j--){
                L->data[j+1]=L->data[j];
            }
            L->data[i-1]=x;
        }
    }
}
```

```

        L->last = L->last+1;
    }
}
return 1;
}

```

按位删除

```

int Delete(sequenlist *L,int i){
    int j;
    if((i<1)|| (i>L->last+1)){
        printf("error\n");
        return 0;
    }else{
        for(j=i;j<=L->last;j++){
            L->data[j-1]=L->data[j];
        }
        L->last--;
    }
    return 1;
}

```

算法分析

插入与删除的时间复杂度为 $O(n)$

链表

可以灵活的进行插入和删除，充分利用存储空间。

数据类型

```

typedef char datatype;
typedef struct node{
    datatype data;
    struct node *next;
} linklist;

```

创建链表

头插法

```

linklist *CreatListF(){
    char ch;
    linklist *head,*s;
    head=NULL;
    while((ch=getche())!="#"){
        s = (linklist*)malloc(sizeof(linklist));
        s->data = ch;
        s->next=head; // 顺序相反的原因
        head=s;
    }
    return head;
}

```

时间复杂度:On(n)

尾插法

```
linklist *CreatListR(){
    char ch;
    linklist *head,*s,*r;
    head=NULL;
    r=NULL;
    while((ch=getche())!="#"){
        s = (linklist*)malloc(sizeof(linklist));
        s->data = ch;
        if(head==NULL)
            head = s;
        else
            r->next=s;
        r=s;
    }
    if(r!=NULL)
        r->next=NULL;
    return head;
}
```

时间复杂度:On(n)

带头节点的尾插法

```
linklist *CreatListR1(){
    char ch;
    linklist *head,*s,*r;
    head=(linklist*)malloc(sizeof(linklist));
    r=head;
    while((ch=getche())!="#"){
        s = (linklist*)malloc(sizeof(linklist));
        s->data = ch;
        r->next=s;
        r=s;
    }
    if(r!=NULL)
        r->next=NULL;
    return head;
}
```

时间复杂度:On(n)

单链表的查找

按序号查找

```

linklist *Get(linklist *head,int i){
    int j;
    linklist *p;
    p=head;j=0;
    while((p->next!=NULL)&&(j<i)){
        p=p->next;
        j++;
    }
    if (i==j) return p;
    else return NULL;
}

```

时间复杂度:On(n)

按值查找

```

linklist *Locate(linklist *head,datatype key){
    linklist *p;
    p=head->next;
    while((p!=NULL)&&(p->data!=key))
        p=p->next;
    if(p==NULL)
        return NULL;
    else
        return p;
}

```

第六章 图

图是一种非线性数据结构，有多个直接前驱和后继。

一个无向图，它的顶点数 n 和边数 e 满足 $0 \leq e \leq n(n-1)/2$ ，如果 $e=n(n-1)/2$ ，则该无向图称为完全无向图。

一个有向图，它的顶点数 n 和边数 e 满足 $0 \leq e \leq n(n-1)$ ，如果 $e=n(n-1)$ ，则称该有向图为完全有向图。

若图中的顶点为 n ，边数为 e ，如果 $e < n \log n$ 则为稀疏图，否则为稠密图。

若路径中的顶点不重复出现，则这条路径被称为简单路径，起点和终点相同并且路径长度不小于2的简单路径，被称为简单回路或者简单环。

在一个有向图中若存在一个顶点 v ，从该顶点有路径可到达图中其他所有的顶点，则称这个图为有根图， v 称为该图的根。

在无向图 G 中，若顶点 $v_i, v_j (i \neq j)$ 有路径相通，则称 v_i 和 v_j 是相通的，如果 G 中的任意两个顶点都连通，则称 G 为连通图；否则为非连通图。无向图 G 中的极大连通子图称为 G 的连通分量。对于任何连通图而言，连通分量就是其自身；非连通图可多个连通分量。

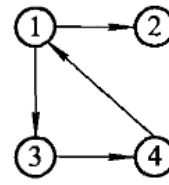
G 是连通图最少有 $n-1$ 条边， G 是非连通图最多有 C_{n-1}^2 条边

在有向图 G 中若从 v_i 到 $v_j (i \neq j)$ 和从 v_j 到 $v_i (i \neq j)$ 都存在路径，称 $v_i, v_j (i \neq j)$ 是强连通的。若 G 中任意两个顶点都是强连通的，则称该图为强连通图。有向图中的极大连通子图称作有向图的连通分量。

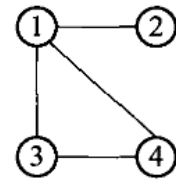
图的存储实现

邻接矩阵

$$A_1 = \begin{bmatrix} 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 \end{bmatrix}, A_2 = \begin{bmatrix} 0 & 1 & 1 & 1 \\ 1 & 0 & 0 & 0 \\ 1 & 0 & 0 & 1 \\ 1 & 0 & 1 & 0 \end{bmatrix}$$



(a)有向图 G_1



(b)无向图 G_2

数据类型

```
#define N 8 //图的顶点数
#define E 10 //图的边数
typedef char vextype; //顶点的数据类型
typedef float Adjtype; //边的权值
typedef struct{
    vextype vexts[N]; //顶点数组
    Adjtype arcs[N][N]; //
} Graph;
```

无向网络建立邻接矩阵

```
// 无向网络建立邻接矩阵
void CreatGraph(Graph *g){
    int i,j,k;
    float w;
    for(i=0;i<N;i++){
        g->vexts[i] = getchar();
        for(j=0;j<N;j++){
            for(k=0;k<E;k++){
                g->arcs[i][j] = 0;
            }
        }
    }
    for(k=0;k<E;k++){
        scanf("%d%d%f",&i,&j,&w);
        g->arcs[i][j] = w;
        g->arcs[j][i] = w;
    }
}

// 建立无向图
void CreatGraph(Graph *g){
    int i,j,k;
    int w;
    for(i=0;i<N;i++){
        g->vexts[i] = getchar();
        for(j=0;j<N;j++){
            for(k=0;k<E;k++){
                g->arcs[i][j] = 0;
            }
        }
    }
    for(k=0;k<E;k++){
        scanf("%d%d%d",&i,&j,&w);
```

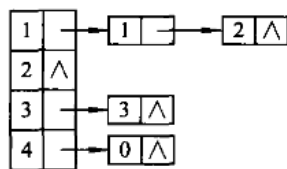
```

        g->arcs[i][j] = w;
        g->arcs[j][i] = w;
    }
}
// 建立有向图
void CreatGraph(Graph *g){
    int i,j,k;
    int w;
    for(i=0;i<N;i++){
        g->vexs[i] = getchar();
        for(j=0;j<N;j++){
            g->arcs[i][j] = 0;
        }
    }
    for(k=0;k<E;k++){
        scanf("%d%d%d",&i,&j,&w);
        g->arcs[i][j] = w; // 写入邻接矩阵
    }
}

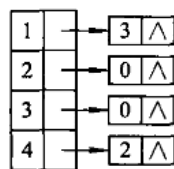
```

当邻接矩阵是一个稀疏矩阵时，会造成空间的浪费，时间复杂度 $O(n^2)$ ，求边的数目的时间复杂度为 $O(n^2)$

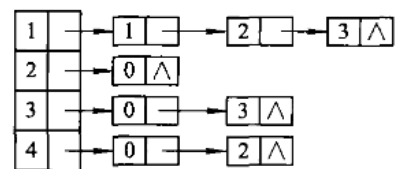
邻接表



顶点表 出边表
(a) 有向图邻接表



顶点表 入边表
(b) 有向图逆邻接表



顶点表 边表
(c) 无向图邻接表

数据类型

```

#define N 8 //图的顶点数
#define E 10 //图的边数
typedef char vextype; //定义顶点数据信息类型
typedef struct node{
    int adjvex; //邻接点域
    struct node *next; //链域
} Edgenode;
typedef struct{
    vextype vertex; //顶点域
    Edgenode *link; //指针域
} vexnode;

```

无向图建立

```
void CreatAdjlist(Vexnode ga[]){
    int i,j,k;
    Edgenode *s;
    for(i=0;i<N;i++){
        ga[i].vertex = getchar();
        ga[i].link = NULL;
    }
    for(k=0;k<E;k++){
        scanf("%d%d",&i,&j);    //读入边
        s = (Edgenode *)malloc(sizeof(Edgenode));
        s->adjvex = j;
        s->next = ga[i].link;
        ga[i].link = s;
        s = (Edgenode *)malloc(sizeof(Edgenode));
        s->adjvex=i;
        s->next = ga[j].link;
        ga[j].link = s;
    }
}
```

有向图建立

```
void CreatAdjlist(Vexnode ga[]){
    int i, j, k;
    Edgenode *s;
    // 初始化顶点
    for (i = 0; i < N; i++) {
        ga[i].vertex = getchar(); // 输入顶点信息
        ga[i].link = NULL;        // 邻接表的指针初始化为 NULL
    }
    // 输入边的信息并构建有向图的邻接表
    for (k = 0; k < E; k++) {
        scanf("%d%d", &i, &j); // 输入一条边 i -> j
        // 为边 i -> j 创建一个新的邻接点节点
        s = (Edgenode *)malloc(sizeof(Edgenode));
        s->adjvex = j;           // 邻接点为 j
        s->next = ga[i].link;    // 将当前顶点 i 的邻接点插入链表头部
        ga[i].link = s;         // 更新顶点 i 的邻接表指针
    }
}
```

上述算法的时间复杂度为 $O(n + e)$ ，判断 (v_i, v_j) 是不是图中的一条边，最坏的情况是 $O(n)$

图的遍历

深度优先搜索遍历

邻接矩阵

```

int visited[n]; //visited为全局变量, n为顶点数
Graph g;
void DFSA(int i){
    int j;
    printf(" node:%c\n", g.vext[i]);
    visited[i] = 1;
    for(j=0; j<n; j++)
        if((g.arcs[i][j]==1)&&(visited[j]==0)))
            DFSA(j);
}

```

时间复杂度 $O(n^2)$,空间复杂度 $O(n)$, 生成序列不唯一

邻接表

```

int visited[n];
Vexnode ga[n];
void DFSL(int i){
    Edgenode *p;
    printf(" node:%c\n",ga[i].vertex);
    visited[i] = 1;
    p=ga[i].link;
    while(p!=NULL){
        if(visited[p->adjvex]==0)
            DFSL(p->adjvex);
        p=p->next;
    }
}

```

时间复杂度 $O(2e + n)$,空间复杂度 $O(n)$, 生成序列不唯一

广度优先搜索遍历

邻接矩阵

```

int visited[n];
Graph g;
SequenQueue *Q;
void BFSA(int k){
    int i,j;
    SetNull(Q);
    printf("%c\n",g.vexs[k]);
    visited[k] = 1;
    EnQueue(Q,k); //访问过的顶点入队
    while(!Empty(Q)){
        i=DeQueue(Q); //出队
        for(j=0; j<n; j++)
            if((g.arcs[i][j]==1)&&(visited[j]!=1)){
                printf("%c\n",g.vexs[j]);
                visited[j] = 1;
                EnQueue(Q,j); //访问过的顶点入队
            }
    }
}

```


使用队列辅助进行遍历

邻接表

```
int visited[n];
Vexnode ga[n];
SequenQueue *Q; //Q为顺序队列
void BFSL(k){
    int i;
    Edgenode *p;
    SetNull(Q);
    printf("%c\n", ga[k].vertex);
    visited[k] = 1;
    EnQueue(Q, k);
    while(!Empty(Q)){
        i=DeQueue(Q);
        p=ga[i].link;
        while(p!=NULL){
            if(visited[p->adjvex] != 1){
                printf("%c\n", ga[p->adjvex].vertex);
                visited[p->adjvex] = 1;
                EnQueue(Q, p->adjvex);
            }
            p=p->next;
        }
    }
}
```

使用队列辅助进行遍历